Setting Up Playwright

Before we write a single test, we need to get Playwright installed and talking to a browser. This guide walks through that setup one step at a time. If a step doesn't work for you, stop and flag it. We'll fix it together rather than push ahead.

> **A note before you start: our corporate setup is a little different**
>
> Normally, when someone installs Playwright, it downloads its own private copies of the Chrome, Firefox, and Safari engines to test against. Our IT security policy blocks that download.
>
> That's not a problem. Playwright has a built-in option to use the Chrome or Edge browser that's already installed on your laptop instead of downloading its own. We'll use that option for this entire class. The setup steps below already account for this, so just follow them in order and you'll be set.
>
> **One side effect to expect:** because we're using your real Chrome or Edge, a visible browser window will pop open every time you run a test. That's normal and actually makes it easier to see Playwright in action.

Step 1: Get the Project Folder

Clone or download the class repository, then open that folder in VS Code.

```
git clone https://github.com/yithril/Dexian_TypescriptAndPlayWright.git
```

Then open the resulting folder in VS Code (File → Open Folder).

Step 2: Install Project Dependencies

From the terminal, inside the project folder, run:

```
npm install
```

This reads the project's package list and installs everything it needs, including Playwright itself. You'll see a progress bar and a folder called `node_modules` appear. This is expected and you can ignore its contents.

Step 3: Install Playwright, Skipping the Blocked Download

This is the one step that's different from a normal Playwright setup. Normally you'd run `npx playwright install`, which tries to download browser engines, the part our network blocks. Instead, we set one switch that tells Playwright "skip the download, I'll use my own browser," and then run install as usual. Type both lines into your terminal, one after the other, then press Enter after each:

```
$env:PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD=1
```

```
npx playwright install
```

The first line tells Playwright not to attempt the blocked download. The second line then installs the rest of what it needs, skipping straight past that step. This only needs to be done once per terminal session, if you close and reopen your terminal later, just run both lines again.

> **Using a different terminal?**
>
> The command above is written for PowerShell, which is what VS Code opens by default on Windows, the terminal most of you will be using. If you're in a different one, use the matching line instead:
>
> **Command Prompt (cmd.exe):** `set PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD=1`
>
> **Mac or Linux Terminal:** `export PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD=1`

Not sure which terminal you're in? Look at the top of the terminal panel in VS Code, it will say `powershell` or `cmd`. When in doubt, raise your hand and we'll check together.

Step 4: Point Playwright at Your Real Chrome or Edge

Open the file named `playwright.config.ts` in the project. We've already set this up for the class, but here's what to look for so you understand what it's doing:

```
use: {
  channel: 'chrome',   // or 'msedge' if you prefer Edge
  // ...other settings
}
```

`channel: 'chrome'` is the line doing all the work here. It tells Playwright "don't look for your own downloaded browser, launch the real Chrome that's already on this laptop instead." If you'd rather use Edge, change it to `'msedge'`.

Step 5: Run the Sample Test

Let's confirm everything works end to end. Run:

```
npx playwright test
```

Here's what you should see happen:

1. A Chrome (or Edge) window briefly pops open on screen.
2. It navigates somewhere and performs a few actions automatically. You're not driving the mouse, the script is.
3. The window closes on its own.
4. Back in the terminal, you'll see a summary showing the test passed.

If Something Goes Wrong

"executable doesn't exist" or a browser download error

This usually means the `PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD` line in Step 4 was skipped, used the wrong syntax for your terminal type, or was run in a different terminal window than the install command. Double check the terminal type and re-run both lines together in the same window, and confirm `channel: 'chrome'` is present in the config from Step 5.

Nothing happens, or the terminal just hangs

Give it a few seconds, the first run is often slower. If it sits for more than 30 seconds, raise your hand.

"command not found" for npm, npx, or git

This points to Node.js or Git not being installed correctly. Flag this early rather than troubleshooting alone, since the fix usually involves an IT-approved installer.